



Claude Code

전사 세미나

클로드코드 잘 사용하기

AI와 함께하는 새로운 개발 워크플로우

발표자

AI 1팀 김영동 대리

일시

2026년 4월 7일

CLAUDE CODE SEMINAR

목차

- 01 코딩은 어디로 가고 있는가
- 02 왜 Claude Code인가
- 03 AI 시대의 개발 방법론
- 04 하네스 & 컨텍스트 엔지니어링
- 05 이렇게 하면 망한다 — 한계와 실패 패턴
- 06 멀티 에이전트 활용
- 07 실전 워크플로우 & 도구 세팅
- 08 비개발자도 할 수 있다
- 09 AI 시대, 우리는 어떻게 해야 하나
- 10 마무리 & Q&A

01

코딩은 어디로 가고 있는가

추상화의 역사 — 1단계

```
01001000 01100101 01101100  
01101100 01101111
```



```
printf("Hello");
```

C 언어

*"기계를 직접 다루지 않으면
개발자가 아니다"*

추상화의 역사 — 2단계

C

```
ptr = malloc(size);  
// ... use ptr ...  
free(ptr);
```



Java

```
Object obj = new Object();  
// ... use obj ...  
// GC handles cleanup
```

"메모리 직접 관리 안 하면 개발이 아니다"

추상화의 역사 — 3단계

```
print("Hello")
```

~~"장난감 언어, 실제 개발에 못 쓴다"~~

Python → #1 언어

추상화의 역사 — 그리고 지금

"이 함수를 리팩토링하고 테스트를 작성해줘"

~~"아건 진짜 개발아 아니다"~~

매번 틀렸다

25%

YC 2025 겨울 배치 중
코드베이스의 95%를 AI로 생성

~3개월

Meta 엔지니어가 코드를
직접 타이핑하지 않은 기간

100%

AI가 짠 코드가 사람의 터치 없이
PR에 그대로 올라오는 일상

SECTION 1

마크다운이 코드를 대체하는 현실

이름은 다르지만 역할은 동일 — 마크다운으로 프로젝트 규칙을 AI에게 전달



Claude Code

`CLAUDE.md`



Cursor

`Cursor Rules`



GitHub Copilot

`copilot-instructions.md`



Codex

`AGENTS.md`

개발자의 새로운 역할

코드를 잘 짜는 능력



문서를 잘 쓰는 능력



컨텍스트를 설계하는 능력

건축가

벽돌 쌓기 대신 설계도 작성.
정확한 설계도 → 튼튼한 건물.

영화 감독

직접 연기 대신 배우에게 디렉션.
CLAUDE.md = 대본

그래도 기초는 중요하다

*"AI가 더 많이 해줄수록,
기초 지식을 가진 사람의 경쟁력이 역설적으로 올라간다"*

변하는 것

직접 코드를 타이핑하는 비중

데모까지는 쉽지만, 실서비스로 만드는 건 전혀 다른 레벨

변하지 않는 것

시스템을 이해하는 능력

보안, 동시접속, DB 스키마, 캐싱 — 기본기 없이는 질문조차 불가

02

왜 Claude Code인가

SECTION 2

AI 코딩 도구의 진화 3단계

1단계

자동완성

IDE가 다음 줄을 예측
반복 패턴에 탁월

한계: 현재 파일로 컨텍스트 제한

2단계

대화형 채팅

코드 토론, 버그 설명
스니펫 붙여넣기

한계: 다중 파일 조율이 수동

3단계

에이전틱 코딩

목표를 분해하고 독립 실행
전체 코드베이스 이해

← 지금 여기

개발 패러다임 전환

기존 개발
코드 작성
↓
테스트 실행
↓
오류 읽기
↓
수정
↓
반복...

에이전틱 개발

목표 정의



변경 검토



구현 승인

에이전틱 코딩의 실제 성과

2주

CTO 추정 4~8개월 프로젝트를
에이전틱 코딩으로 완료

Augment Code + Vertex AI

1~2일

신규 개발자 온보딩
기존 수주~수개월에서 단축

첫날부터 의미 있는 기여 가능

복합효과

불가능하던 작업 실현
기술 부채 해결 · 즉각 피드백

SECTION 2

AI 코딩 도구 비교

도구	형태	특징
GitHub Copilot	IDE 플러그인	코드 자동완성 중심, 가장 대중적
Cursor	IDE	AI 네이티브 에디터, 에이전틱 기능 내장
Windsurf	IDE	에이전틱 IDE, Cascade 에이전트
Codex (OpenAI)	터미널 CLI	에이전틱, 클라우드 샌드박스 실행
Claude Code	터미널 CLI	에이전틱, 로컬 실행, 확장 레이어 구조

SECTION 2

CLI 기반 — IDE에 종속되지 않음

터미널이면 어디서든 실행. 스크립트화, 자동화, CI/CD 통합에 유리.



터미널 CLI

핵심 인터페이스



VS Code

확장 프로그램



JetBrains

확장 프로그램



데스크톱 앱

Mac / Windows



웹

claude.ai/code



Chrome 확장

브라우저 통합

에이전트 루프

목표만 주면 작업 완료까지 알아서 반복. 언제든지 중단하여 방향 수정 가능.

1

컨텍스트 수집

파일 검색, 코드 구조 파악
설정 파일 읽기, 의존성 매핑



2

작업 수행

파일 편집, 셸 명령 실행
새 파일 생성



3

결과 검증

테스트 실행, 빌드 확인
오류 분석 → 1로 복귀

SECTION 2

내장 도구 — 5가지 범주, 20개+

파일 작업

파일 읽기, 코드 편집, 새 파일 생성, 이름 변경 및 재구성

검색

패턴으로 파일 찾기, 정규식으로 콘텐츠 검색, 코드베이스 탐색

실행

셸 명령 실행, 서버 시작, 테스트 실행, git 사용

웹

웹 검색, 문서 가져오기, 오류 메시지 조회

코드 인텔리전스

타입 오류/경고 확인, 정의로 이동, 참조 찾기

확장 레이어 구조

처음에는 CLAUDE.md만으로 시작. 필요에 따라 하나씩 추가.

CLAUDE.md	모든 세션에서 로드되는 컨텍스트	여기서 시작
Skills	재사용 가능한 워크플로우	반복 작업이 생길 때
MCP	외부 서비스 연결 (100개+)	Notion, GitHub, Slack 등
Subagents	격리된 컨텍스트, 요약 반환	복잡한 조사/병렬 작업
Agent Teams	여러 독립 세션이 협업	대규모 병렬 작업
Hooks	이벤트 기반 자동화 스크립트	링크, 테스트 자동화
Plugins	기능 패키징 및 배포	팀/커뮤니티 공유

Subagent vs Agent Team

둘 다 작업을 위임하는 방식. 규모와 독립성에 따라 선택.

Subagent

구조

메인이 생성한 격리된 워커

컨텍스트

자체 윈도우, 요약만 반환

통신

주 에이전트에게만 보고

많은 파일 읽기, 병렬 조사

Agent Team

구조

여러 독립 세션이 협업

컨텍스트

완전히 독립, 각자 별도 윈도우

통신

팀원끼리 직접 메시지

보안/성능/테스트 동시 검토

빠르고 효율적

Sonnet

대부분의 코딩 작업에 적합

빠른 응답 속도

일상적인 개발 워크플로우

기본 모델

깊고 정교한

Opus

복잡한 아키텍처 결정

깊은 분석이 필요한 경우

대규모 리팩토링

/model 로 즉시 전환

가장 좋은 모델부터. 성능이 곧 결과물의 품질.

권한 모드와 안전장치

권한 모드 — Shift+Tab으로 전환

default

각 도구 처음 사용 시 권한 요청

acceptEdits

파일 편집 자동, 명령은 요청

plan

읽기 전용 — 분석만 가능

auto

배경 안전 검사로 자동 승인

안전장치

체크포인트

모든 파일 편집은 되돌릴 수 있음

Esc 두 번으로 복원

도구별 승인

읽기 전용 → 자동 허용

Bash/파일 수정 → 승인 필요

세밀한 규칙

/permissions로 Allow/Ask/Deny 설정

deny가 항상 최우선

왜 생태계가 중요한가

도구의 기능 → 시간이 지나면 추격 가능.

커뮤니티가 만든 플러그인·스킬·노하우·교육 자료 → 복제 불가.

문제가 생겼을 때 검색하면 답이 나오는 환경

= 실무 도입의 핵심 조건

03

AI 시대의 개발 방법론

SECTION 3

왜 방법론이 필요한가

에이전틱 코딩 등장 → 병목의 이동.

이전의 병목

코드를 작성하는 것

지금의 병목

AI에게 **무엇을** 시킬지 정하는 것
+ 정말 원하는 대로 했는지 **확인**하는 것

타임라인

2025.02

Karpathy, "Vibe Coding" 제시

AI 코딩의 대중화, 문화적 전환점

2025 중반

AI 생성 코드의 품질/보안 문제 부각

구조화된 방법론의 필요성 대두

2025 하반기

SDD, 스펙 주도 프레임워크 등장

Vibe Coding의 한계에 대한 업계 대응

2026 초

TDD, Context Engineering 부상

성숙한 AI 개발 워크플로우의 정립

구조화 수준 스펙트럼

왼쪽일수록 자유 · 오른쪽일수록 엄격.

VDD

바이브 코딩

탐색적

PDD

프롬프트 주도

의도

TDD

테스트 주도

검증

SDD

스펙 주도

계획

VDD — 바이트 코딩

AI에게 '분위기'만 전달 · 생성 코드 검토 생략 · 결과 그대로 수용 — 이런 개발 방식.

"저는 이것을 '바이트 코딩'이라 부릅니다. 분위기에 완전히 몸을 맡기고, 기하급수적 성장을 받아들이며, 코드가 존재한다는 사실 자체를 잊는 것입니다."

— Andrej Karpathy, 2025.02

적합한 경우

프로토타이핑, MVP

주말 사이드 프로젝트

아이디어 탐색

Collins Dictionary 2025 올해의 단어
검색량 6,700% 급증

현실적 문제

AI 생성 코드의 ~45%에 보안 결함

"3개월 블랙박스" — 아무도 설명 못 함

"시작하는 데는 괜찮지만 유지보수에는 끔찍"
— Linus Torvalds

PDD — 프롬프트 주도 개발

프롬프트를 위시리스트가 아닌, 검증 가능한 명세(contract)로 취급.

좋은 PDD 프롬프트

```
"장바구니 할인 계산 함수를 구현해줘.  
- 입력: items[{price, quantity}],  
      coupon_code(optional)  
- 규칙: 5만원 이상 → 무료배송,  
      미만 → 배송비 3,000원  
- 출력: { subtotal, discount,  
      shipping_fee, total }  
- 검증: 'SUMMER10' → 10% 할인"
```

"할인 기능 만들어줘" ← 나쁜 예

장점

자연어로 접근성 높음
비개발자도 참여 가능
프롬프트 자체가 문서 역할

한계

좋은 프롬프트 작성 능력 필요
복잡한 시스템에서 충분한 명세 어려움

인간 → 테스트 작성 · AI → 통과 코드 구현.

1. [인간]

실패 테스트(Red) 작성

2. [AI]

테스트를 통과하는 최소 코드(Green)

3. [인간]

통과 코드 리뷰

4. [AI]

통과 상태 유지하며 리팩토링

TDD 지시 예시

"장바구니 할인 계산을 TDD로 구현해줘.

먼저 테스트부터 작성해:

- 기본 합계: 10,000원 × 2개 = 20,000원
- 배송비: 5만원 미만 → 3,000원,
이상 → 무료
- 쿠폰: 'SUMMER10' → 10% 할인

테스트가 실패하는 걸 확인하고,

그 다음에 통과하는 코드를 작성해."

핵심: 인간이 테스트, AI가 구현 · 한 번에 하나 · 테스트 수정 금지

버그 **40~80%** 감소 — 2025 DORA

AI가 TDD를 치팅하는 3가지 방식

"The genie doesn't want to do TDD." — Kent Beck. 프로세스 강제 장치 필요.

관찰되는 3가지 치팅 패턴

1. 테스트 삭제/skip/xfail

구현이 막히면 테스트 자체를 비활성화

2. 구현 먼저, 테스트 나중

자기 코드에 맞춘 약한 테스트를 작성

3. assert 약화

값 검증을 타입 검증으로, Equal→True로 슬쩍

강제력의 스펙트럼

약	프롬프트
↓	문서 (CLAUDE.md)
↓	슬래시 커맨드 / 스킵
↓	에이전트 격리
강	자동 차단 혹은

개인 프로젝트 = 문서로 충분 · 조직 표준 = 혹 필요.

SDD — 스펙 주도 개발

코드가 아닌 스펙이 단일 진실 원천(Source of Truth). Vibe Coding 이후 업계의 궤도 수정.

워크플로우

[인간] 요구사항을 구조화된 스펙으로 작성

[AI] 스펙을 읽고 구현 계획 제안

[인간] 계획 검토 및 승인

[AI] 스펙에 따라 코드 생성

[자동화] 스펙 준수 여부 검증

스펙 예시

기능: 장바구니 할인 계산

요구사항

- 상품 목록(price, quantity) 합계 계산
- 쿠폰 코드 적용 시 할인을 차감
- 5만원 이상 무료배송, 미만 3,000원

제약조건

- 쿠폰 1회만 적용 가능
- 할인 후 금액 음수 불가

완료 기준

- [] 정상 합계 계산
- [] 유효 쿠폰 할인 반영
- [] 만료 쿠폰 → COUPON_EXPIRED
- [] 5만원 이상 → 무료배송

SECTION 3

SDD 실전 — 3단계 워크플로우

도구는 달라도 성숙한 SDD 도구들이 공통으로 취하는 패턴 — 각 단계가 **파일로 저장 · 커밋 · 여러 세션 참조**.

1. SPECIFY

WHAT · WHY

요구사항, 사용자 스토리, 수용 기준. **HOW는 금지.**

2. PLAN

HOW

기술 스택, 아키텍처, 데이터 모델. 조직 원칙 게이트 통과.

3. TASKS

분해 · 매핑

병렬 가능한 작업 단위로 분해. 작업 ↔ 요구사항 매핑 유지.

장치: **[NEEDS CLARIFICATION]** 마커 — SI의 모호한 부분 추측 차단. 마커 발견 → 구현 중단, 사람에게 확인.

SECTION 3

방법론 비교

	Vibe	PDD	TDD	SDD
인간의 역할	묘사자	프롬프트 작성자	테스트 작성자	스펙 작성자
검증 방식	감각적	출력 비교	자동 테스트	스펙 검증
진입 장벽	매우 낮음	낮음	중간	중간~높음
프로토타이핑	매우 빠름	빠름	중간	느림
프로덕션	낮음	중간	높음	높음
유지보수	매우 낮음	중간	높음	높음

결론

Spec + TDD

Spec

"무엇을" 정의
의도 명확화

+

TDD

"정말 그렇게 됐는지" 검증
결과를 자동으로 확인

스펙 없는 TDD — 나무는 보되 숲은 놓침

TDD 없는 스펙 — 검증 수단 부재

그런데 이걸 매번 수동으로 할 것인가?

→ 스펙 템플릿을 계획 문서로 정착시키고

→ Skills로 **TDD 워크플로우**를 패키징하고

→ Hooks로 **테스트 자동 실행**을 걸면

이 시스템이 곧 **하네스 엔지니어링**

04

하네스 & 컨텍스트 엔지니어링

엔지니어링의 진화

2023-2024

프롬프트 엔지니어링

모델에

무엇을 **말할** 것인가

단일 프롬프트 · 정적

2024-2025

컨텍스트 엔지니어링

모델에

무엇을 **보여줄** 것인가

입력 파이프라인 전체 · 동적

2025-2026

하네스 엔지니어링

모델 주변에

무엇을 **구축할** 것인가

실행 환경 전체 · 시스템 아키텍처

SECTION 4

여전히 살아있는 프롬프트 엔지니어링

용어가 "컨텍스트 엔지니어링"으로 옮겨간 것뿐, 사라진 것이 아님. **가장 안쪽 층엔 여전히 프롬프트.**

"프롬프트 엔지니어링은 죽지 않았습니다. 흡수되었을 뿐입니다. 좋은 컨텍스트 엔지니어는 여전히 좋은 프롬프트 엔지니어입니다."

— Phil Schmid, Hugging Face

프롬프트

한 번 말하는 법

컨텍스트

여러 프롬프트가
모이는 구조

하네스

그 구조 주변의
실행 환경

안쪽 영성 → 바깥도 붕괴. 모호한 CLAUDE.md 한 줄 → 스킴·혹을 붙여도 결국 모호한 결과.

왜 프롬프트만으로는 부족한가

좋은 프롬프트로도 잔존하는 문제. 에이전트가 여러 단계를 거치는 순간 — 숫자가 불리해짐.

재현성 문제

같은 프롬프트에도 다른 결과.
AI는 비결정적 시스템.

복리로 쌓이는 실패

한 단계 5% 실패율이
20단계에선 치명적.

숫자로 보면

각 단계 성공률 **95%**
20단계 체인이라면?

36%

$$0.95^{20} = 0.358$$

"95% 동작" 에이전트 →
실제 작업의 2/3에서 실패.

SECTION 4

Agent = Model + Harness

AGENT
에이전트

=

MODEL
텍스트 생성

+

HARNESS
나머지 전부

"모델이 아닌 것은 전부 하네스입니다." — LangChain, Vivek Trivedy

비유 · 마구(馬具, horse tack)

고삐 · 안장 · 재갈처럼, 강력하지만 예측 불가능한 말(= LLM)을 올바른 방향으로 이끄는 장비 전체가 하네스.

무엇을 보는지
컨텍스트

할 수 있는지
도구 · 권한

언제 멈추는지
제약

잘못됐을 때
복구

하네스의 6대 구성 요소

1 컨텍스트 엔지니어링

각 단계에서 모델이 보는 정보를 설계. 무엇을 유지·요약·버릴지 관리

2 도구 오케스트레이션

어떤 도구가 가용한지, 실행이 어떻게 처리되는지 결정

3 상태 및 메모리

수 분~수 시간 실행되는 에이전트의 내구성 있는 상태 관리

4 검증 루프

출력 생성 후 행동 실행 전 발동. 스키마·의미적·정책 유효성

5 오류 복구

실패 감지 → 재시도, 다른 접근법, 인간에게 폴백

6 인간 개입 제어

핵심 의사결정에서 일시정지. DB 삭제? 결제? → 승인 요구

SECTION 4

에이전트 루프 — 하네스의 심장

거의 모든 코딩 에이전트가 반복하는 4단계. 각 단계가 하네스의 설계 지점.

1. GATHER CONTEXT

파일 읽기, 검색, 질문

무엇을 **안 보여줄지**를 설계하는 자리. 노이즈 유입 → 4가지 실패 모드 발동.

2. TAKE ACTION

코드 작성, 명령 실행, 툴 호출

가능한 한 **되돌릴 수 있는** 액션으로. 실패 비용 낮음 → 과감함 허용.

3. VERIFY WORK

테스트, 린트, 타입체크, 리뷰

자동 피드백 부재 → 루프가 **환각 강화**. 이 단계 막히면 전부 붕괴.

4. REPEAT

결과를 다음 루프 컨텍스트로

이전 결과가 다음 루프에 주입되는 지점. **Compaction-Pruning**이 필수.

하네스 엔지니어링 = 이 4단계의 각 지점을 신뢰성 있게 만드는 작업.

SECTION 4

컨텍스트의 4가지 실패 모드

"컨텍스트 윈도우가 크니까 많이 넣어도 OK" 상식 — 사실 아님. (* 컨텍스트 윈도우 = 모델이 한 번에 읽을 수 있는 텍스트량. 요즘 모델은 대략 20만 토큰, 한국어 기준 약 15만 자) Drew Breunig가 정리한 4가지 실패 모드.

POISONING · 중독

환각이 한 번 들어오면 이후 계속 참조되며 오류 누적

→ 존재하지 않는 함수를 반복 호출

DISTRACTION · 산만

히스토리에 과의존, 과거 행동을 반복

→ 새 전략을 못 세우고 실패한 액션 재시도

CONFUSION · 혼동

무관한 정보까지 "고려해버린다"

→ 토큰 30개 이상이면 선택 실패 급증

CLASH · 충돌

모순된 정보가 있으면 추론이 무너진다

→ 초반 오답 1개로 평균 39% 성능 저하

"컨텍스트를 잡동사니 서랍처럼 다루면, 그 잡동사니가 답변에 영향을 줍니다." — Drew Breunig

핵심은 "뭘 넣을까"보다 "뭘 뺄까".

SECTION 4

Context Rot — 길수록 조용히 썩는다

Chroma Research가 Claude · GPT · Gemini · Qwen 등 **18개 최신 모델**을 대상으로 측정한 결과.

50k 토큰

20만 토큰 모델이 이미 열화되는 지점
한국어 약 4만 자. 윈도우가 짝 차기 훨씬 전부터 성능 하락.

방해문장 1개

하나만 섞어도 성능 저하
4개면 급감. "많이 넣으면 좋다" 직관 붕괴.

"관련 정보가 컨텍스트에 있느냐가 전부는 아닙니다. 더 중요한 것은 그 정보가 **어떻게** 제시되느냐입니다."

— Chroma Research

컨텍스트 = 길이가 아닌 **신호 대 잡음비**. 쓸 수 있는 만큼 다 채울 필요 없음.

SECTION 4

Right Altitude — 지시문의 고도

CLAUDE.md나 프롬프트를 쓸 때 가장 흔한 실수는 두 극단.

너무 낮음 · OVER-SPECIFIED

모든 케이스를 if/else로 박기

→ 예외 상황에서 brittle

너무 높음 · UNDER-SPECIFIED

"잘 해줘", "깔끔하게"

→ 모델이 추측, 세션마다 결과 다름

올바른 고도 공식

원칙 1줄 + 이유(why) 1줄 + 예시 1~2개

이게 붙어야 새 상황에도 일반화 가능.

자가진단 2문항:

- ① 이 규칙이 내가 의도하지 않은 상황에서도 말이 되는가? → 되면 OK
- ② 이 규칙을 아무 세션에 붙여도 행동이 바뀌지 않는가? → 그러면 너무 높음

SECTION 4

Long-Horizon 3기법

오래 실행되는 에이전트(수십 분~수 시간) — 컨텍스트 예산 금방 고갈. Anthropic이 정리한 3가지 처방 — 공통 원리는 **컨텍스트를 바깥으로 덜어내기**.

1. COMPACTION

요약본으로 리셋

아키텍처 결정·핵심 사실은 보존 · 탐색 로그는 폐기.

2. NOTE-TAKING

외부 파일에 기록

`NOTES.md` , `PROGRESS.md` . 세션 단절에도 이어서 작업 가능.

3. SUB-AGENT

서브에이전트 격리

각자 컨텍스트에서 작업 · 1~2k 토큰 요약만 반환.

Claude Code 구현: `/compact` · 프로젝트 파일 읽기/쓰기 · `Task` 서브에이전트

SECTION 4

Claude Code의 4개 레이어

정구봉(Team Attention)이 제시한 에이전트 시대의 Context Engineering 레이어



CLAUDE.md

시스템 성격 정의

프로젝트 규칙, 컨벤션,
아키텍처 지침



Skills

실행 로직 구조화

재사용 가능한 워크플로우,
슬래시 커맨드



Hooks

이벤트 기반 자동화

결정론적 스크립트,
12+ 이벤트 타입



MCP

외부 도구 연결

Notion, GitHub, Slack 등
10,000+ 활성 서버

CLAUDE.md — 프로젝트의 "헌법"

모든 세션이 시작될 때 자동으로 로드되는 지속 컨텍스트. 4개 레이어 중에서도 **가장 먼저 손대야 하는 레이어**.

먼저 짚고 갈 것 — "지시가 아니라 컨텍스트다"

많은 사람이 CLAUDE.md를 시스템 프롬프트로 오해. 사실은 시스템 프롬프트 **다음의** 사용자 메시지로 주입. 즉, 강제(enforcement)가 아닌 **권고(context)**. 모호 → Claude의 임의 해석.

✓ 검증 가능 · 구체적

- TypeScript strict mode
- 커밋 전 `pnpm lint && test`
- **any** 타입 금지
- 세션은 Redis (TTL 30분)

× 모호 · 검증 불가

코드를 잘 작성해주세요.

테스트를 해주세요.

좋은 코드를 만들어주세요.

결정적 차이: 나쁜 예는 **지위도 행동 불변** — 한 줄 삭제 테스트 실패. (→ 다음 페이지)

SECTION 4

CLAUDE.md 운영 원칙

공식 문서가 느낌표까지 붙여서 경고한 유일한 원칙 — **짧을수록 효과적.**

한 줄 삭제 테스트

"각 줄에 대해 물어보세요 — 이 줄을 지우면 Claude가 실수할까? 아니면 잘라내세요."

— Claude Code Best Practices

크기 가이드라인

~60줄 · HumanLayer (엄격파)

2.5k 토큰 · Boris Cherny 팀

200줄 이하 · 공식 문서

500줄 이하 · Marconato

숫자보다 **신호 대 잡음비**. 같은 신호 → 짧을수록 우세.

비대해지면 — 모듈화

`@path/to/file` — 다른 파일을 import (최대 5홉)

`.claude/rules/` — 경로별 규칙, 읽을 때만 트리거

`/compact` 후에도 CLAUDE.md는 재로드

자가진단 5문항: ① 한 줄 삭제 테스트 ② 린터로 잡을 규칙은 린터로 ③ `/compact` 생존 테스트 ④ 모순 테스트 ⑤ 되묻기 테스트

SECTION 4

CLAUDE.md는 살아있는 문서 — 현장 사례

한 번 쓰고 끝나는 정적 문서가 아닌, **피드백으로 자라는 유기체**처럼 운영.

Boris Cherny

Claude Code 창시자 · Anthropic

주 수 회 업데이트. **Claude가 실수할 때마다 한 줄 추가**. PR에서 @claude 태그 → GitHub Action이 CLAUDE.md 자동 커밋.

Kieran Klaassen

Self-Improving Harness

"마음에 안 드네"/"좋은 지적이다" 한 마디가 곧 새 규칙. 백그라운드 잡 rate limit 버그 1회 → 규칙 1줄 → **재발 0건**.

정구봉 (Team Attention)

규모의 실천

CLAUDE.md 159개 + 스킬 100개+ 동시 운영. "파일 하나는 짧게, 대신 파일을 많이."

Hunslog

한국 커뮤니티 · 미니멀

2.5k 토큰 이내, **반복 실수만** 기록. 몇 주에 한 번 Claude에게 CLAUDE.md 자체를 리뷰시키는 메타 운영.

"버그는 한 번 고치는 게 아니라, CLAUDE.md에 한 줄을 추가하면서 고치는 것이다."

Skills & Hooks

Skills — 워크플로우 패키징

```
---  
name: deploy  
description: Production deploy  
---  
  
1. pnpm test  
2. pnpm lint  
3. Create release tag  
4. Push to main
```

점진적 공개 (Progressive Disclosure)

세션 시작: 메타데이터만 로드 (수십 토큰)

관련 감지 시: 전체 지시문 로드

Hooks — 이벤트 기반 자동화

LLM이 아닌 결정론적 스크립트

12+ 이벤트 타입:

SessionStart, PreToolUse,
PostToolUse, Notification ...

파일 편집 후 → **ESLint 자동 실행**

커밋 전 → **테스트 자동 실행**

.env 편집 시도 → **차단**

작업 완료 → **Slack/Telegram 알림**

SECTION 4

Plugins — 하네스를 포장해서 팀 자산으로

4개 레이어 위에 얹히는 배포 레이어 — 2025.10 공개 베타

새 기능이 아니라 포장지

이미 존재하는 **Skill · Subagent · Hook · MCP · LSP**를 단일 단위로 번들링해 팀·커뮤니티에 배포하는 메커니즘

```
my-plugin/  
├── .claude-plugin/plugin.json  
├── commands/ agents/  
├── skills/    hooks/  
└── .mcp.json
```

standalone → **plugin** 전환은 파일 이동 + 매니페스트 1개 수준

마켓플레이스로 공유

```
# 마켓플레이스 추가  
/plugin marketplace add <repo>  
  
# 플러그인 설치  
/plugin install name@marketplace
```

개인 **.claude/**에 쌓다가
동료에게 공유하고 싶어지는 순간 전환

개인이 쌓아올린 하네스를
조직 자산으로 전환하는 마지막 한 조각

SECTION 4

Plan-Critic-Build — 계획과 실행의 분리

에이전트가 가장 많이 실패하는 지점: **계획 없이 바로 코드 작성**. 해결책은 단순 — 계획 단계를 쓰기 권한 없는 모드로 분리.

1. PLAN

읽기만 · 쓰기 금지

파일 읽기·검색·질문만. 결과: 편집 가능한 계획 문서.

2. CRITIC

다른 눈으로 본다

리뷰 전용 에이전트. 작성자의 blind spot을 드러내기.

3. BUILD

승인 후 실행

승인된 계획 링크만 컨텍스트로, 쓰기 권한 개방.

핵심: Planner와 Critic을 같은 컨텍스트에 두지 않기. 자기 계획 → 자기 비평 시 blind spot 잔존.

Claude Code의 Plan Mode(`Shift+Tab` `Shift+Tab`) — 1단계를 구조적으로 강제.

"계획과 실행을 분리하는 것이 제가 AI 코딩에서 하는 가장 중요한 행위이다." — Armin Ronacher

SECTION 4

Ralph Loop — 가장 단순한 하네스

Ralph Loop = 단 하나의 **PROMPT.md** 파일을 **bash** 무한 루프로 반복 실행하는 가장 원시적인 에이전트 패턴. 매 루프마다 에이전트가 현재 상태에서 "다음 한 걸음"을 자가 선택. Geoffrey Huntley가 정식화한 철학.

```
while ;; do
  cat PROMPT.md | claude-code
done
```

왜 동작하는가

PROMPT.md에 "가장 중요한 일 하나"만 적는다

실수하면 도구 탓 말고 규칙을 추가한다

단순성이 예측 가능성을 만든다

솔직한 한계

Greenfield 전용. 레거시에 부적합

검증(테스트/리트) 없으면 환각을 쌓음

튜닝하는 엔지니어는 여전히 필요

"놀이터에서 아이가 미끄럼틀 위에서 뛰어내려 다치면, '뛰지 말고 미끄러져 내려오세요' 표지판을 세웁니다." — Huntley

암묵지를 파일로 뽑아내라

"AI에게 뭘 보여줄지 어떻게 알지?" → 답은 이미 **여러분 머릿속**. 리뷰에서 "이건 이렇게 하는 게 낫다"고 말하던 것, 버그 한 번 겪고 반사적으로 체크 하던 것 — 그걸 파일로 추출.

① 결정의 "이유"를 쓴다

결과만 → 금방 낡음. 이유까지 → 새 상황에도 일반화.

② Post-Mortem을 규칙으로

"계획과 실제 구현의 분기점?" → 한 줄 규칙으로 승화.

③ Self-Improving

"마음에 안 드네", "좋은 지적이다" 피드백마다 규칙 한 줄씩 성장.

④ 취향을 전용 에이전트로

글로 쓰기 어려운 취향은 Simplification / Security / Style 에이전트로 encode.

"버그는 한 번 고치는 게 아니라, *CLAUDE.md*에 한 줄을 추가하면서 고치는 것이다."

같은 실수 반복 없는 시스템.

SECTION 4

Harness Builder — 새로운 역할

Builder

기능을 구현하고
코드를 작성

Reviewer

코드의 품질, 정확성,
보안을 검토

Harness Builder

에이전트가 일하는
환경 자체를 설계

OpenAI Codex 팀 — 5개월간 수동 코드 0줄로 100만 줄 프로젝트

"우리 팀의 주된 업무는 에이전트가 유용한 일을 할 수 있도록 환경을 만드는 것이 되었다."

1 리포지토리 지식 체계 설계

2 결정론적 제약 구현 (린터, 테스트)

3 에이전트 가독성 최적화

4 취향과 원칙의 인코딩

05

이렇게 하면 망한다

한계와 실패 패턴



SECTION 5

신뢰성의 수학 — 복리로 쌓이는 실패

각 단계 성공률 **95%** 가정 → 다단계 작업의 전체 성공률?

단계 수	전체 성공률
1 단계	95%
5 단계	77%
10 단계	60%
20 단계	36%
50 단계	8%

계획 → 탐색 → 작성 → 테스트 → 수정 → 커밋 — 20단계만 가도
2/3이 실패

에이전트를 만드는 건 쉬운 부분 — 프로덕션 신뢰성이 진짜 엔지니어링

검증 루프 · 재시도 · 체크포인트 — 복합 실패율을 낮추는 건 **하네스**
스뿐

실패 패턴 5가지

도구를 잘 쓰는 것만큼 중요한 건, **잘못 쓰는 패턴**을 아는 것.

01 컨텍스트 과부하

"많이 주면 잘하겠지" → **섹션 4** 참고

02 불명확한 요구사항

"이거 좀 고쳐줘" → LLM은 학습 데이터 기반 **추측 구현**. 맞아도 재현성 없음

03 검증 없는 자동수락

"AI가 만들었으니 맞겠지" — 가장 위험한 생각. 자기 작업 평가 시 **병리적 낙관 주의자**

04 긴 세션 드리프트

시점 편향 · 오류 중첩 · Mode Collapse — 초반 규칙을 잊고 방향 상실

05 권한 과다

"바이패스가 편하니까" — 프롬프트 인젝션 · 토큰 유출 · 프로덕션 사고의 단일 경로

검증 없는 자동수락 — Anthropic 내부 사고

외부 해킹이 아닌 — 에이전트의 과잉행동에서 온 사고. Anthropic이 공개한 실제 사례.

사고 1

원격 브랜치 삭제

"clean up branches"를 해석해 원격 브랜치를 잘못 지움

사고 2

인증 토큰 업로드

엔지니어의 GitHub 토큰을 내부 컴퓨터 클러스터에 업로드

사고 3

프로덕션 DB 마이그레이션

테스트 DB로 착각하고 프로덕션에 마이그레이션 실행 시도

대안 Diff 리뷰 → 테스트 실행 → 생성자/평가자 분리(GAN 스타일) → 중요 작업 승인 게이트

위협 모델 — 에이전트가 위험한 4가지 이유

Anthropic이 분류한 에이전트 위협 모델 — 기술이 아니라 **경계**의 문제.

OVEREAGER · 과잉행동

목표는 이해했지만 승인 범위를 넘는 주도성

→ "clean up" 한마디에 원격 브랜치 전부 삭제

HONEST MISTAKES · 실수

영향 범위를 잘못 판단

→ 테스트 DB인 줄 알고 프로덕션에 실행

PROMPT INJECTION · 주입

파일·웹페이지·도구 출력에 심어진 악의적 지시

→ 읽은 문서의 숨은 지시가 파일 삭제를 유도

MISALIGNED · 정렬 문제

모델이 자체 목표를 추구 — 이론적 위험

→ 현재는 관찰되지 않지만 설계 시 가정

"can we fix this?"는 **질문**이지 지시가 아니다.

사용자가 승인한 것과 에이전트가 추론한 것의 경계를 명확히.

그래도 여전히 중요한 기초

"AI가 다 해주니까 코드 안 배워도 되겠다"

— 가장 위험한 생각

AI의 자신감 있게 틀린 코드 제시 — 기초 없으면 틀림 감지 불가

추상화 수준만 올라갔을 뿐, 문제 해결 · 시스템 사고 · 디버깅은 여전히 필수

진입장벽은 낮아졌으나, 고품질 역량은 오히려 더 구조화

번역기의 비유

자동 번역기가 아무리 좋아져도 번역 품질 판단에는 원문 언어 이해 필수. AI 코딩 도구도 마찬가지 — 정확성·보안성·성능을 평가할 눈이 없으면 코드를 떠안는 것.

06

멀티 에이전트 활용

단일 에이전트의 세 가지 벽

하나의 에이전트가 오래 일할수록 필연적으로 맞닥뜨리는 벽 — **쪼개야 할 이유**.

01 컨텍스트의 벽

파일 읽고 테스트 돌리고 로그 수집하는 동안 컨텍스트가 쓰레기로 찬다.

50k 토큰부터 Context Rot · Distraction · Confusion 발동

02 역할의 벽

"쓰고, 구현하고, 리뷰하고, 리팩터하라"를 한 에이전트에게 → **전부 어설프게**.

자기 코드 자기 리뷰 = blind spot 그대로

03 신뢰성의 벽

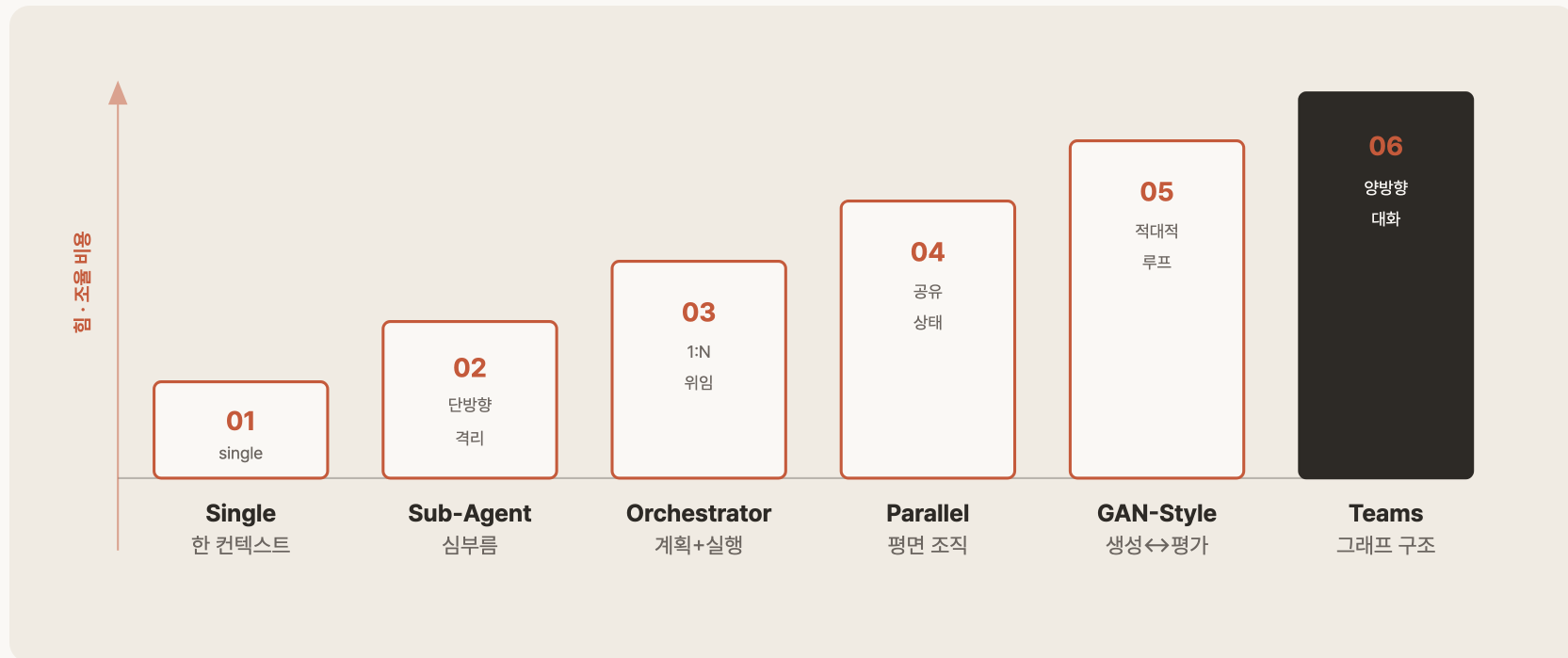
단계별 95% × 20단계 연쇄 = **전체 성공률 36%**.

긴 체인 = 수학적 실패

하나의 에이전트 = **하나의 일**.

진화의 스펙트럼

멀티 에이전트는 하나의 패턴이 아니라 **점점 복잡해지는** 층위다.



SECTION 6

핵심 패턴 5가지

무엇을, 어떻게 **쫄개느냐**에 대한 서로 다른 답.

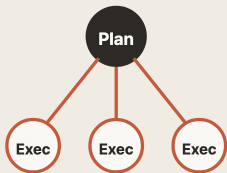
01 Sub-Agent



단방향 심부름

탐색 · 검증 · 로그 파싱

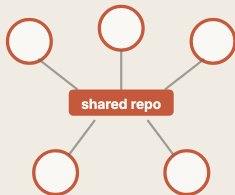
02 Orchestrator



1:N 위임

계획자 + 실행자들

03 Parallel



평면 조직

Git worktree 격리

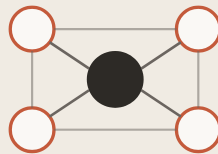
04 GAN-Style



적대적 루프

루브릭으로 수렴

05 Agent Teams



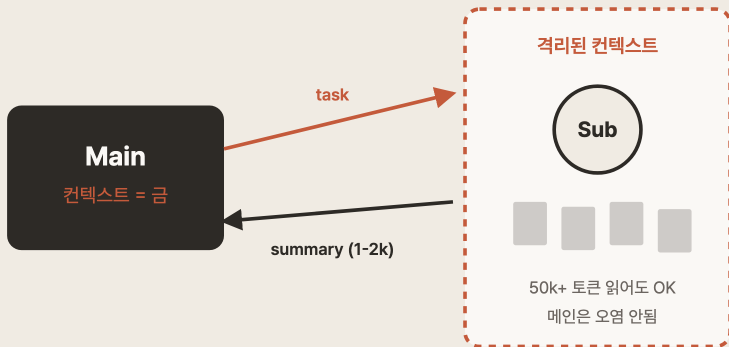
양방향 대화

그래프 구조

SECTION 6

Sub-Agent — 가장 자주 쓰는 패턴

가장 **과소평가**된 패턴 — 일상 작업의 80%는 여기서 해결.



컨텍스트 격리

파일 50개 뒤져도 메인 오염 없음

도구 제한

읽기 전용만 허용 → 사고 원천 차단

정보 격리 = 치팅 차단

Tester는 구현 못 봄 · Impl은 스펙 못 봄

"메인 에이전트의 컨텍스트는 **금**이다. 서브에이전트가 만드는 것은 **요약**이다."

SECTION 6

Agent Teams — 양방향 대화가 가능한 팀

Sub-Agent가 "시켜놓고 받는" 구조라면, Teams는 **같이 토론하고 합의하는** 구조.

동적 팀 구성 예 — 경쟁사 분석



자동 수정 루프

Worker 구현 → QA 검증 → Support 수정 → QA 재검증

공유 컨텍스트 बैं크

프로젝트 스펙·제약은 중앙에 두고 참조

관찰 가능성 필수

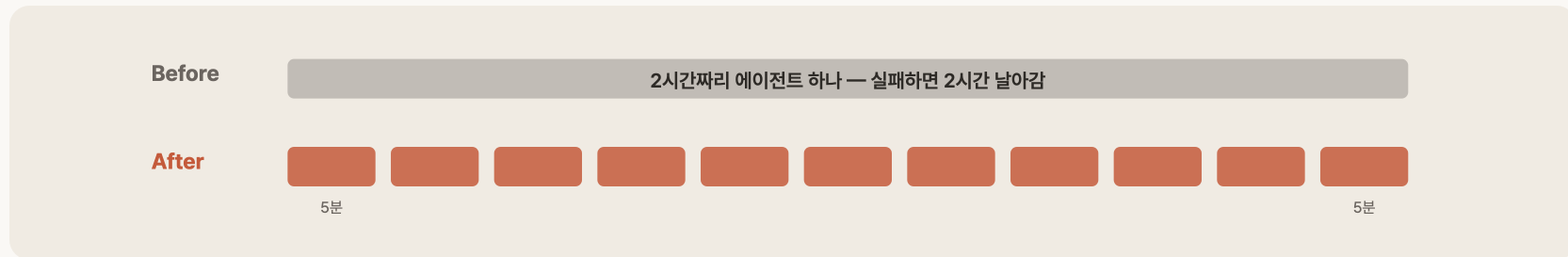
팀원 간 대화를 Slack·Discord로 미러링

3명까지는 이득 · 10명 넘으면 대기 시간 > 작업 시간 — **"팀이면 다 좋다"는 환상 경계.**

SECTION 6

5분 에이전트 철학

긴 에이전트 하나보다 **짧은 에이전트 여럿**. 가장 반직관적인 교훈.



실패 지점이 짧다

5분 실패 = 5분 손해

컨텍스트 신선

50k 넘기 전에 끝남

재시도 가능

역등성 설계 쉬움

관찰 가능

입출력으로 판단

"5분짜리 에이전트 여럿을 합치면 — 장시간 에이전트 없이 장시간 시스템이 나온다."

— Paul Fruitful (Claude Code 하네스 분석)

설계 4대 원칙

패턴이 무엇이든 공통으로 지켜야 하는 것 — **쪼갬의 네 축**.

01

컨텍스트 격리

각 에이전트 = 자기 일에 필요한 것만 · 메인 = 결정과 요약만.

02

역할 경계 명시

"무엇을 하고 무엇을 **안** 하는지" 명시 · 모호함 → 중복과 공백.

03

사전 모호성 제거

실행 후 발견 → 이미 늦음. Ralpton 우승팀 — 개발 전 **133회 Q&A**.

04

검증 주체 분리

생성자 ≠ 검증자. Self-review → blind spot 잔존.

멀티 에이전트 = **"더 많은 AI"가 아니라 "더 좁은 역할의 AI 여럿"**

컨텍스트를 쪼갬다 · 역할을 쪼갬다 · 검증을 쪼갬다 · 시간을 쪼갬다

07

실전 워크플로우 & 도구 세팅

SECTION 7

솔직한 고백 — 다 지키는 건 불가능

지금까지 다룬 것 — CLAUDE.md 3층위, Skill·Hook, 서브에이전트, Plan Mode, Worktree, 멀티 에이전트, 컨텍스트 방어, 암묵지 문서화, 모델 라우팅...

직접 다 세팅한다

- 한 번 세팅해도 업데이트 안 됨
- 팀에 공유되지 않음
- 몇 달 뒤 본인도 기억 못 함
- → **번아웃**

원리 없이 플러그인만

- 이상 동작 시 원인 판단 불가
- 같은 실수를 반복
- 디버깅할 수 없음
- → **블랙박스 의존**

원리는 이해 · 구현은 위임 — 잘 만들어진 플러그인 활용.

OMC — 원리를 명령어로

Oh My Claude Code — 한국 개발자 Yeachan Heo 제작, GitHub Trending 1위. 발표 4~6 섹션 원리의 거의 전부를 한 플러그인으로 패키징.

"Don't learn Claude Code. Just use OMC."

원리 → OMC 구현 매핑

발표에서 다룬 원리

멀티 에이전트 역할 분리
모델 라우팅 · 비용 최적화
Plan-Critic-Build
병렬 에이전트
검증 루프 · GAN 스타일
모호성 제거 · 소크라테스 질문
Worktree 병렬 · 관찰 가능성

OMC 구현

29+ 사전 정의 에이전트 (architect-critic-tester ...)
Haiku/Sonnet/Opus 자동 선택 — 토큰 **30~50% 절감**
`/ralplan` — architect 계획, critic 별도 비평
`/ultrawork`, `/ultrapilot` — 최대 5 워커 동시
`/ultraqa` — test → verify → fix 자동 순환
`/deep-interview` — 요구사항 인터뷰
`/project-session-manager` + HUD + Slack 알림

OMC의 3 레이어 — 실행 모드 · 에이전트 · 모델

실행 모드

같은 작업도 모드에 따라 다르게 동작

- /autopilot — 완전 자율
- /ultrapilot — 5위커 병렬
- /ralph — 지속 루프 + 검증
- /team — 구조화 파이프라인
- /ultraqa — QA 집중
- /ecomode — 비용 최적

에이전트 29+

역할·권한·모델이 다른 전문가

- 분석 · architect, analyst, tracer
- 실행 · executor, designer, writer
- 탐색 · explore, explore-high
- 검증 · critic, code-reviewer, security-reviewer, qa-tester
- 특수 · debugger, scientist, vision

모델 라우팅

난이도에 맞춰 자동 선택

Haiku

탐색 · 단순 변경

Sonnet

표준 구현 · 리팩토링

Opus

아키텍처 · 비평

사용자는 "어떤 작업인지"만 말한다 — 모드·에이전트·모델 선택은 OMC가 알아서.

SECTION 7

5개 명령어로 돌리는 Agent Loop

섹션 4 Agent Loop의 다섯 단계 — 각각 **하나의 명령어**로 매핑.

1. 이해

/deep-dive

코드베이스 Trace → 소크라테스식 인터뷰 → **머릿속이 파일로 떨어진다**

2. 계획

/ralplan

architect 계획 → critic 별도 비평 → **합의된 것만 실행 허가**

3. 실행

/team

워커 4~5개 병렬 · Test Writer는 구현 코드 못 봄 · **역할 격리 기본**

4. 검증

/ultraqa

test → 원인 분석 → 수정 → 재실행 · **루프가 알아서 수렴**

5. 학습

/learner

재사용 가능한 패턴을 **새 스킬로 자동 저장** — 쓸수록 똑똑해짐

SECTION 7

한 걸음 더 — 단일 모델 탈피

섹션 6이 여러 Claude였다면, 이젠 여러 종류의 AI. 모델마다 상이한 훈련 분포·blind spot.

/ask [provider]

특정 공급자에게 바로 질의 — Claude 답이 확신 안 설 때 다른 모델에 의견

```
# 성능 조인을 Codex에게  
/ask codex "이 함수 최적화"  
  
# 복잡도 분석을 Gemini에게  
/ask gemini "시간 복잡도"
```

/ccg — Claude · Codex · Gemini

세 모델에 동시 송출 → 답을 모아 합의·불일치·불확실 분리 표시

특히 빛나는 순간

- 아키텍처 결정 (되돌리기 어려움)
- 디버깅이 막혔을 때
- 머지 전 코드 리뷰 최종 확인

주의 — 비용 3배, 속도는 가장 느린 모델 기준, 합의가 증명은 아니다(같은 편향 공유 가능). **중요한 결정에만.**

OMC도 해결하지 못하는 것

OMC는 마법이 아님. 섹션 5의 실패 패턴은 플러그인 레이어에서도 여전히 유효.

원리 모르면 블랙박스

결과가 이상할 때 **왜 이상한지** 판단하려면 섹션 4~6의 원리 이해가 필수.
OMC는 원리를 **대체**하지 않고 **가속**한다

프로젝트 맥락은 여전히 사용자 몫

CLAUDE.md · 팀 규칙 · 이슈 트래커 · MCP 세팅은 OMC가 모름. "이 프로젝트의 맥락"은 직접 주입 필요

컨텍스트 실패 모드는 그대로

Poisoning · Distraction · Confusion · Clash는 플러그인 레이어에서도 발동. 긴 세션은 **주기적으로 끊기**

플러그인 의존 리스크 · 비용

업데이트로 동작 변경 · 깨짐 가능. "**플러그인 없이도 돌아가는 최소 경로**"를 알 것. 병렬 Opus는 싸지 않음

플러그인 = 원리의 **가속기**, **대체재**가 아님.

플러그인 고르는 체크리스트

1 원리가 보이는가

문서를 읽고 어떤 원리 구현인지 설명 가능? 못 하면 블랙박스 의존

2 깎 수 있는가

플러그인 없이도 같은 작업이 (느리더라도) 가능? 종속이 과하면 위험

3 관찰 가능한가

뒤에서 뭘 하는지 로그·HUD·알림으로 보이는가

4 유지되는가

마지막 커밋이 언제? 생태계는 빨리 움직인다

5 내 병목에 맞는가

"멋있어 보이는 것"이 아니라 실제로 자주 하는 작업의 시간을 줄이는가

한 장 요약

원리를 이해한 사람만이 좋은 플러그인을 제대로 활용 가능.

- 매일 손으로 다 세팅은 불가능
- 커뮤니티가 이미 패키징해둠
- OMC는 그중 대표

하지만 — 원리, 맥락, 최소 경로는 여전히 사용자 책임

08

이 발표는 처음부터
직업 이야기가 아니었다

증거부터 — 지금 보고 있는 이 장표

이 96장의 슬라이드 — 발표자 1인이 Claude와 함께 만들었다. 그리고 **이 작업에는 코드가 한 줄도 없다.**

파이프라인 — 3층 + 1 Skill

research/ · official/ · blog/ · linkedin/ · youtube/

↓ 원자료 수집

sections/XX/README.md

↓ 계획 · 서술

decks/.../slide-XX.html

↓ **slides-grab** Skill

claude-seminar.pdf

+ **CLAUDE.md** — 디자인 규칙·카피 원칙 박제 (하네스)

앞 섹션들에서 본 원리 그대로
Spec-first · 하네스 · Skill 재사용

특별한 기술 없음
마크다운 + HTML + 규칙 문서

이 섹션의 진짜 질문 — **그럼 앞의 모든 섹션은 누구의 이야기였나.**

재료 1 — 폴더 구조로 박힌 분리

수집층 · 계획층 · 실행층이 **폴더 경계로 분리**. 규칙으로 지키려 하지 않고 파일 구조가 강제.

```

claude-seminar/
├─ research/ # 하네스·컨텍스트 리서치
├─ official/ # Claude Code 공식 문서
├─ blog/ # Anthropic + 한국 커뮤니티
├─ linkedin/ # 실무자 인사이트
├─ youtube/ # 영상 요약
├─
├─ sections/ # 9개 섹션 마크다운 (계획)
├─ decks/claude-seminar/ # 슬라이드 (실행)
│   └─ slide-01.html ~ slide-96.html
├─
└─ CLAUDE.md # 프로젝트 규칙 (하네스)
  
```

수집층 = 근거의 정박지

에이전트에게 "이 주장 어디서?" 물으면 파일로 답이 돌아온다

계획층 ≠ 실행층

sections/에 본문 먼저, decks/는 디자인만 — 섹션 4의 Plan-Critic-Build 강제

근거 없이 슬라이드 작성 → **환각이 장표로 굳는다.**

자료 2 — CLAUDE.md, 취향의 박제

디자인 감각·카피 원칙이 문서로 굳어져 있다. 한 번 박히면 새 슬라이드가 자동으로 따른다 — Self-Improving Harness의 가장 원시적 형태.

CLAUDE.md 발췌 — 카피 원칙

1. 명사형·구 단위만 — "계획과 실행의 분리"
 2. 금지: ~다 / ~이다 서술형
 3. 금지: ~합니다 / ~입니다 공손체
 4. 금지: ~하라 / ~해주세요 명령형
 5. 금지: ~을까 / ~인가 수사 의문형
- # 예외: 인용문·다크카드 슬로건·제목

디자인 토큰 (6개)

```
bg #faf8f5 / card #f0ebe3
dark #2d2a26 / accent #c45a3b
720pt × 405pt, 16:9
```

처음부터 있던 규칙이 아니다

초반: ~합니다 공손체 → 스캔 불가

중반: ~다 서술형 → 여전히 읽어야

결론: 명사형 통일 → CLAUDE.md 박제

96장에 걸쳐 카피 톤이 일관된 이유 —

사람이 96번 검토해서가 아니라, **규칙이 한 번 박혔기 때문.**

SECTION 8

재료 3 — slides-grab, Skill 경계로 박힌 분리

HTML → PDF 익스포트를 직접 짜지 않는다. 커뮤니티 Skill을 그대로, 그리고 의도적으로 **3단으로 분리**.

STAGE 1 · PLAN

slides-grab-plan

`slide-outline.md` 생성

사용자 승인까지 반복

STAGE 2 · DESIGN

slides-grab-design

승인된 아웃라인 →

`slide-XX.html` 생성·수정

STAGE 3 · EXPORT

slides-grab-export

HTML → PDF 변환

산출물 검증

왜 3단인가 — 단계 사이에 사용자 승인을 **물리적으로 강제**. Plan에서 방향이 틀어지면 Design·Export 비용 전부 낭비.

섹션 4의 "**계획과 실행의 분리**"가 Skill 경계로 박제됨 — 규칙이 아니라 파일 구조가 강제.

SECTION 8

사실 이 발표는 처음부터 직업 이야기가 아니었다

지금까지 다룬 것들을 쭉 나열해 보자.

프롬프트를 잘 쓰는 법

컨텍스트를 설계하는 법

하네스를 만드는 법

에이전트에게 역할을 나눠주는 법

계획 · 실행 · 검증을 분리하는 법

암묵지를 파일로 뽑아내는 법

직접 만들지 말고 잘 만든 것을 쓰는 법

이 중에 **"개발자에게만"** 해당하는 게 있는가? 하나도 없다.

전부 도구를 다루는 보편 원리 예시가 마침 코드였을 뿐.

원칙은 그대로, 예시만 교체

앞 섹션의 원칙을 **코드가 아닌 업무**로 바꿔 읽어보자. 원칙은 그대로, 예시만 다르다.

발표의 원칙

Spec-first

CLAUDE.md

Plan Mode 먼저

검증 주체 분리

Context Rot 방어

암묵지 명시화

멀티 에이전트

업무로 번역

시작 전에 원하는 것을 **글로 먼저 쓴다**. 채팅창에서 즉흥 금지

내 업무의 규칙·용어·판단 기준을 **하나의 파일로**

실행 전에 "무엇을 어떻게 할지" **계획부터**

초안 쓴 에이전트와 **검토하는 에이전트를 분리**

주제가 바뀌면 **새 세션**. 한 창에 섞지 않기

머릿속 판단 기준을 **예시와 함께 글로**

큰 일을 **작은 역할로 쪼개서** 맡긴다

변한 것은 **예시뿐**. 원칙은 그대로. — 새로 배울 것이 없다.

진짜 차이는 하나 — 안전 감각

개발자는 `git reset` 이 반사적으로 나온다. 비개발자에게는 이 감각이 처음 생긴다 — 역량 차이가 아니라 **경험 차이**.

비개발자에게 더 크게 다가오는 것

- "동작하는 것 같음"과 "실제 맞는 것"의 구별
- 실패의 결과가 눈에 안 보임 (덮어쓴 파일, 잘못 나간 메일)
- 권한의 의미가 직관적이지 않음

의식적인 안전장치 4가지

1. 권한은 항상 **확인형** — 자동 실행 금지
2. 외부에 내보내기 전 **사람 검수**
3. 중요한 작업은 **Plan Mode**로 계획부터
4. 이상하면 즉시 멈춘다

이 넷만 지켜도 실패의 **80%**가 사라진다 — 앞에서 이미 다룬 원칙, 더 의식적으로.

"글쓰기가 새로운 코딩" — 진짜 대상

AI 시대의 시니어리티는 자기 일을 글로 설명할 수 있는 능력이다.

이 능력을 이미 가진 사람들

PM — 신입 교육 문서를 쓴다

디자이너 — 브랜드 가이드를 쓴다

오퍼레이션 — SOP를 쓴다

모두 자기 업무의 판단 기준을 글로 뽑는 훈련을 이미 받았다.

역전의 가능성

"글쓰기로 일을 잘하는 사람"은 AI 시대에 개발자보다 먼저 출발선에 서 있을 수도 있다.

가장 희소한 자원은 "무엇을 원하는지 정확히 말할 줄 아는 능력". 코딩은 그 능력의 한 형태였을 뿐.

시작하는 사람에게 — 직업 무관 4가지

1 자기 일에서 반복을 찾는다

주 1회 이상 손으로 하는 일

2 그걸 글로 쓴다

무엇 · 왜 · 어떻게 · 주의점

3 작게 시작한다

완벽한 자동화가 아니라 "한 단계 덜 해도 되는 것"부터

4 결과를 검증한다

외부로 나가는 것은 반드시 사람이 확인

"컨텍스트를 설계하는 사람이 이긴다" — 개발자와 비개발자 모두에게 **똑같이** 적용된다.

09

AI 시대, 우리는 어떻게 해야 하나

이 두려움은 처음이 아니다 — 1956년

70년 전, 컴파일러가 등장했을 때 어셈블리 프로그래머들이 **정확히 같은 말**을 했다.

1956년의 말들

"컴파일된 코드는 수작업 어셈블리만큼 효율적일 수 없다."

"메탈까지 내려가지 않으면 무슨 일이 일어나는지 어떻게 믿나?"

"쉬워지면 숙련된 프로그래머의 가치가 떨어지지 않을까?"

"프로그래밍의 성직자 계급 — 평범한 인간에게는 너무 복잡한 비밀의 수호자."

— John Backus (FORTRAN 창시자)

고급 언어는 프로그래머 수요를 줄인 게 아니라 **폭발시켰다**.

이 패턴은 반복된다 — **고급언어 · 인터넷 · 모바일 · 클라우드**. 매번 "우리 없어진다"고 했고, 매번 반대 일이 일어났다.

최전선의 피로 — 그들도 똑같이 느낀다

"지식이 쌓이는 속도보다 감가상각되는 속도가 더 빠르다. 나도 지쳤다."

— Simon Willison, 2025

FOMO-AI · 2025 학계 척도

- 뒤처질까 두려움 (backwardness)
- 좋은 도구를 못 쓸까 두려움 (access)
- 남만 이득 볼까 두려움 (dividend)

논문의 결론

FOMO를 가장 크게 줄이는 것은 더 많은 정보가 아니라, **직접 써보는** 것이다.

맨 앞에 있는 사람들조차 똑같이 느낀다 — 다른 점은 "더 빨리 달리기" 대신 "무엇을 남길지 고르기"에 시간을 쓴다는 것뿐.

진짜 두려움의 이름 — 실업이 아니라 정체성

"지금 개발자들이 겪는 것은 기술 위기가 아니다. **정체성 위기**다. 우리는 직업이 아니라 자기 자신을 특정 기술과 동일시해왔다."

— Codelens, *The Developer Identity Crisis Isn't About AI*, 2025

잘못된 질문

"AI가 내 일을 빼앗을까?"

진짜 질문

"실행 자체만으로는 내 가치가 없다면, 나는 **무엇으로** 의
미를 만드는가?"

답을 서두르지 말자. 질문에 **제대로 이름을 붙이는 것**만으로도 많은 것이 바뀐다.

SECTION 9

증폭되는 경험 — 세 개의 증언

50년 넘게 코드를 쓴 사람, 최전선에 있는 사람, 업계의 대부 — 세 사람이 거의 **같은 말**을 한다.

KENT BECK

TDD 창시자 · 52년

"프로그래밍은 여전히 프로그래밍이다. 어떤 면에서는 훨씬 더 나은 경험이다. 시간당 더 **중요한 결정**을 내리게 된다."

SIMON WILLISON

Vibe Engineering

"AI는 **기존 전문성을 증폭**시킨다. 더 많은 기술과 경험을 가질수록 LLM으로부터 더 빠르고 더 좋은 결과를 얻는다."

MARTIN FOWLER

Thoughtworks

"AI는 주니어의 산출물은 복제할 수 있지만, **시니어의 경험과 판단은 복제할 수 없다.**"

AI 시대에 경험의 가치는 떨어지지 않는다. **증폭된다.**

Spec-first · TDD · Plan-Critic · 검증 분리 — 시니어가 원래 하던 것. AI는 그걸 할 줄 아는 사람에게 레버리지를 준다.

처음으로 자본이 되는 당신의 취향

"내가 '이건 싫어' 또는 '좋은 지적이야'라고 말할 때마다, 시스템은 더 똑똑해진다. 조직의 기억이 보존되고 검색 가능해진다."

— Kieran Klaassen, Compound Engineering

지금까지

당신의 취향·판단·도메인 감각은 퇴근하면 사라졌다

아무도 접근할 수 없는 머릿속에만. 전수하려면 수년, 그나마 대부분 유실.

시간 = 노동

이제부터

CLAUDE.md 한 줄 · Skill 한 파일 · 리뷰 규칙 한 줄에 스며든다

매일 조금씩 쌓이고, 다음 날 에이전트가 그것을 기억한다. 한 번 말한 것이 자동으로 적용된다.

시간 = 자본

AI 시대는 당신의 경험을 처음으로 복리로 쌓이는 자산(compounding asset)으로 만든다.

관점 전환 한 문장

나는 AI에게 대체되는 사람이 아니다.

내 경험과 취향이 처음으로 **자본이 되는 시대**에 들어선 사람이다.

이 관점에서 보면

CLAUDE.md = 내 취향을 자본화하는 한 줄

Skill = 내 루틴을 자산으로 승격

서브에이전트 = 내 판단을 복제 가능하게

Plan-Critic = 내 리뷰 감각을 시스템에

한 가지 실용 원칙

"딸깍쇼를 보고 딸깍쇼를 찍지 마라."

직접 써보지 않은 기술에 대해서는 **판단하지 않는다**. 남의 요약본 100개보다 내 손으로 한 번 써본 것이 낫다.

10

마무리 & Q&A



Claude Code

감사합니다

Q&A · 질문 환영
